

Name: Peram Mahitha

Reg no: 192524136

1. CNNs vs. Vision Transformers for Object Detection

Scenario: A self-driving car company needs to detect rare urban obstacles (traffic cones, animals) in real-time using limited labelled datasets.

Questions:

1. Compare CNNs and Vision Transformers in terms of:

- o Feature extraction capabilities for local vs. global context**
- o Performance with small or sparse objects**
- o Computational complexity and inference speed for embedded systems**

2. Discuss which method is better suited for real-time deployment and how transfer learning or data augmentation could impact performance.

1. Comparison of CNNs and Vision Transformers

Factor	CNNs	Vision Transformers (ViTs)
Feature extraction	CNNs use convolution filters to learn local features such as edges, shapes, textures, and object parts. Deeper layers gradually capture larger/global context.	ViTs use self-attention to capture relationships between different parts of an image, giving them strong global context understanding.
Local vs. global context	Excellent at local patterns and spatial details. Global information is obtained through multiple convolution layers.	Excellent at modelling global relationships, even between distant image regions.

Factor	CNNs	Vision Transformers (ViTs)
Small/sparse objects	Generally effective for small objects because convolution preserves spatial information and detects local patterns. Multi-scale CNN detectors can perform well on small obstacles.	Can detect small objects well, but usually need sufficient training data and appropriate image resolution/features.
Limited labelled data	Usually performs better with limited data because CNNs have strong built-in image-processing assumptions such as locality and translation invariance.	Usually requires more data to train effectively from scratch.
Computational complexity	Generally lower computational requirements and efficient on embedded hardware.	Self-attention can be computationally expensive, especially for high-resolution images.
Inference speed	Usually faster and more suitable for real-time applications.	Standard ViTs may have higher latency, although lightweight ViTs are becoming faster.
Embedded systems	Well suited for devices with limited memory and processing power.	More difficult to deploy on resource-constrained embedded systems unless lightweight architectures are used.

2. Which method is better for real-time deployment?

For the given self-driving car scenario, a CNN-based object detection model is generally the better choice when the system has limited labelled data and requires real-time detection.

A CNN-based detector can efficiently identify local features such as the shape, colour, and edges of traffic cones, animals, pedestrians, and other small obstacles. CNN models are also relatively computationally efficient, making them suitable for embedded hardware used in vehicles.

Examples include lightweight CNN-based models such as YOLO variants, SSD, and Mobile Net-based detectors.

However, Vision Transformers have an important advantage in understanding global relationships and scene context. For example, a ViT-based detector may better understand the relationship between an obstacle and its surrounding road environment. Lightweight hybrid CNN-ViT models can therefore provide a balance between local feature extraction and global context.

3. Effect of Transfer Learning

Transfer learning can significantly improve performance, especially because the company has only a small labelled dataset.

A model can first be pretrained on a large dataset such as COCO or ImageNet and then fine-tuned using the company's smaller dataset containing rare urban obstacles.

Benefits include:

- Reduces the amount of labeled training data required.
- Improves detection accuracy.
- Reduces training time.
- Helps recognize common visual features such as edges, textures, and shapes.
- Makes it easier to detect rare objects such as animals or unusual traffic obstacles.

Transfer learning is particularly useful for both CNNs and ViTs, but it is especially important for ViTs because they generally benefit greatly from large-scale pretraining.

4. Effect of Data Augmentation

Data augmentation can increase the diversity of the limited training dataset. For a self-driving car application, useful techniques include:

- Image rotation – handles objects appearing at different orientations.
- Scaling and cropping – improves detection of small and distant objects.
- Horizontal flipping – provides additional training examples.
- Brightness and contrast changes – helps with different lighting conditions.

- Weather augmentation – simulates rain, fog, or low-light conditions.
- Noise addition – improves robustness to camera noise.

For example, if only a few images of traffic cones are available, augmentation can generate different versions of those images, helping the model learn more robust features.

Conclusion

For this scenario, CNNs are the preferred choice for real-time deployment because they provide strong local feature extraction, work well with small datasets, and generally offer faster inference with lower computational requirements. Transfer learning and data augmentation can further improve their performance when labelled data is limited.

Vision Transformers are powerful for capturing global context, but their higher computational requirements and greater dependence on large-scale training data can make standard ViTs less suitable for resource-constrained real-time systems. Therefore, a lightweight CNN or CNN-ViT hybrid model would be a practical solution for detecting rare urban obstacles in a self-driving car.

2. PCA-Based Shape Priors vs. Graph-Based Learning for 3D Shape

Correspondence

Scenario: An animation studio is aligning 3D human face meshes for morphing applications. Meshes vary in topology and vertex density.

Questions:

1. Compare PCA-based shape priors and graph-based learning in terms of:

- o Ability to handle non-rigid deformations**
- o Robustness to varying mesh resolutions and noise**
- o Ease of implementation and computational cost**

2. Analyse which approach would provide more accurate correspondences for realistic morphing and why.

2. PCA-Based Shape Priors vs. Graph-Based Learning for 3D Shape Correspondence

Scenario

An animation studio needs to align **3D human face meshes** for realistic facial morphing. The meshes may have different vertex densities, resolutions, and topologies.

1. Comparison of PCA-Based Shape Priors and Graph-Based Learning

Factor	PCA-Based Shape Priors	Graph-Based Learning
Non-rigid deformations	Works well for common facial variations such as smiling, mouth movement, and changes in facial shape, but mainly captures variations present in the training data.	Better at handling complex and non-rigid deformations because it learns relationships between neighbouring vertices and local/global mesh structures.
Varying mesh resolutions	Usually assumes consistent vertex correspondence and similar mesh structure. Different vertex counts/topologies can make PCA difficult to apply directly.	More flexible because meshes can be represented as graphs. It can learn from different local neighbourhood structures and can be adapted to varying resolutions.
Noise robustness	Relatively robust if noise is not too large because PCA focuses on major variations and ignores some small variations.	Can be robust to noise when trained with noisy examples, but excessive noise can affect graph features and neighbourhood relationships.
Implementation	Simple and easy to understand. Requires constructing a shape graph covariance matrix and computing principal components.	More complex because it requires graph construction, feature extraction, model training, and often neural-network frameworks.
Computational cost	Generally low after the PCA model has been trained.	Usually higher because graph neural networks require more computation during training and inference.

Factor	PCA-Based Shape Priors	Graph-Based Learning
Accuracy	Good for faces that are similar to the training dataset.	Potentially higher accuracy for complex and diverse facial shapes and deformations.

2. Which approach is better for realistic morphing?

For this particular application, graph-based learning would generally provide more accurate correspondences.

The main reason is that a human face contains complex non-rigid deformations. For example, expressions such as smiling, blinking, frowning, and opening the mouth change the positions of many facial regions in different ways. Graph-based learning can model relationships between neighbouring vertices and learn both local geometric features and global facial structure.

PCA-based methods are simpler, but they work best when the meshes have similar topology, vertex count, and consistent correspondence. Since the scenario specifically states that the meshes have different topology and vertex density, PCA becomes less convenient.

Why Graph-Based Learning is More Suitable

Graph-based learning can:

1. Represent a 3D mesh as a graph, where vertices are nodes and mesh connections are edges.
2. Learn relationships between neighbouring facial regions.
3. Handle complex non-rigid facial movements.
4. Capture both local details, such as the eyes and lips, and larger facial structures.
5. Provide better correspondence when the input meshes have different resolutions.
6. Improve alignment accuracy for realistic facial morphing.

Role of PCA

PCA is still useful as a baseline or preprocessing technique. It can provide a compact representation of common facial shapes and is computationally inexpensive. For a controlled

animation pipeline where all meshes have already been registered to the same topology, PCA can be a very practical choice.

Conclusion

Graph-based learning is the better choice for this scenario because the meshes have varying topology and vertex density and need to represent complex non-rigid facial deformations. Although it is more difficult and computationally expensive than PCA, its ability to learn geometric relationships can produce more accurate vertex correspondences and more realistic morphing results.